

Dijkstra's Algorithm

411025020

Questions

1. Analyze the time complexity when implement step 2 by sequential searching whole adjacency list.

假設有 n 個節點，每個節點的鄰接列表平均包含 m 條邊。使用順序搜索整個鄰接列表時，尋找最小橋接邊 e_b 的時間複雜度在最壞的情況下，我們需要遍歷所有未訪問節點的鄰接列表，並找到其中權重最小的邊。因此總時間複雜度為 $O(nm)$ ，對於每個節點需要 $O(m)$ 的時間複雜度來尋找最短邊。

2. Analyze the time complexity when implement step 2 by a priority queue that maintained the bridge edges.

	插入	取出、刪除
double-linked list	$O(1)$	$O(n)$
Min-heap	$O(\log n)$	$O(1)$

1. Implement the priority queue by double-linked list.

有 n 個點，因此時間複雜度為 $O(n^2)$

2. Implement the priority queue by Min-heap.

有 n 個點，因此時間複雜度為 $O(n \log n)$

3. Design an algorithm to reconstruct the path from spanning tree T (the result).

在更新節點距離時同時記錄前一個節點，最後只須從終點循著紀錄的節點即可回溯至起點。

4. How to find the unreachable (have no path from s) set?

紀錄每個節點的訪問情況，執行完Dijkstra演算法後若仍為 *False* 及代表從指定的起點 s 無法抵達此點，這些點的集合即為 unreachable set。

5. Solve <http://oj.csie.ndhu.edu.tw/problem/ALG07C> and analyze your code.

C++

```
#include<iostream>
#include <vector>
#include <queue>
using namespace std;

int main()
{
    int T,n,m,s,t,from,to,w;
    ios_base::sync_with_stdio(false);
```

```

cin.tie(NULL);
cin>>T;
for(int ts=1;ts<=T;ts++)
{
    cin>>n>>m>>s>>t;
    vector<vector<pair<int,int>>> graph(n,vector<pair<int,int>>());
    for(int i=0;i<m;i++)
    {
        cin>>from>>to>>w;
        graph[from].push_back({w,to});
        graph[to].push_back({w,from});
    }
    vector<int> dist(n,200000000);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>> > pq;
    dist[s]=0;
    pq.push({0,s});
    while (!pq.empty())
    {
        auto p=pq.top();
        if(p.second==t)
            break;
        pq.pop();
        if(dist[p.second]>=p.first)
        {
            for(auto path:graph[p.second])
            {
                if(path.first+dist[p.second]<dist[path.second])
                {
                    dist[path.second]=path.first+dist[p.second];
                    pq.push({dist[path.second],path.second});
                }
            }
        }
    }
    if(dist[t]<200000000)
        cout<<"Case #"<<ts<<": "<<dist[t]<<endl;
    else
        cout<<"Case #"<<ts<<": unreachable"<<endl;
}
return 0;
}

```